

How far can r-ary alias trees stretch an OSAM⁺ trace?

Working note (SPARQ-OSAM), September 29, 2026

Abstract

We study the number of reads that the r-ary multi-write pointers of `sam-model` (`RaryPointer`) add to an OSAM⁺ trace. Against an oblivious adversary, who fixes the access schedule in advance, and with a randomized tree layout, we prove three things. The cost of objects with at most b aliases is deterministic. Each dereference deepens any alias by at most one level. The total trace length exceeds its expectation by $O(\sqrt{s \sum_o r_o^2})$ except with probability e^{-s} , where the ranges r_o are computed from the schedule. For the *binary* pointer we prove a deterministic amortized bound, $2 \log_2 f + 1$ reads per dereference plus $O(f)$ once, which holds even against adaptive adversaries. We state the stronger r-ary bound that the experiments support ($O(c\sqrt{sK})$ for K dereferences of high-fan-in objects) as a theorem conditional on a bounded-influence lemma, and record the measurements. Section 7 gives an r-ary splay rule, path halving, with a proven deterministic bound: $2 \log_2 f + 1$ reads per dereference, amortized. It keeps every structural property of the current rule, and the experiments support a $2 \log_b f + O(1)$ bound, which we state as a conjecture.

1 Model

Definition 1 (Alias tree). An object with $f \geq 2$ aliases is a rooted tree whose root holds the value, whose leaves are the aliases, and whose internal nodes have at most b ordered children ($b \geq 2$ even). A single pointer ($f = 1$) is the root itself. The *install* layout is the balanced tree built by `RaryPointer::install`, of height $H_0(f) = \lceil \log_b f \rceil$.

Definition 2 (Dereference). Let alias x be at depth D , on the path $u_0, \dots, u_D = x$ from the root u_0 . A dereference reads the $D + 1$ cells $u_D, u_{D-1}, \dots, u_0$, so its cost is

$$R = \text{dep}(x) + 1.$$

Let A_j be the children of u_j other than u_{j+1} , for $0 \leq j < D$. The path is dissolved and rebuilt as follows:

1. $P \leftarrow (x', A_{D-1})$, where x' is the fresh leaf of x .
2. For $j = D - 2, D - 3, \dots, 0$: $P \leftarrow \text{Pack}(P \circ A_j)$.
3. A fresh root receives the items of P as its children.

`Pack(M)` returns M if $|M| \leq b$. Otherwise it splits M into $m = \lceil |M|/b \rceil$ consecutive chunks of balanced sizes, and puts every chunk of size ≥ 2 under a fresh internal node.

The standalone model in `src/aliasmodel.rs` implements exactly this rule. It reproduces the library's read counts on 150,000 dereferences ($b \in \{2, \dots, 30\}$, $f \leq 3000$, three access orders; `src/bin/validate_model.rs`).

Definition 3 (Randomized layout). The initial assignment of aliases to install leaves is a uniform bijection. Every Pack, and the final placement under the root, first applies a fresh uniform permutation to its items. All other randomness is independent across dereferences.

Definition 4 (Oblivious adversary, trace length). The adversary fixes a schedule $\sigma = ((o_1, a_1), \dots, (o_K, a_K))$ of (object, alias) dereferences before any randomness is drawn. It may depend on public parameters (b and every fan-in f_o) but not on the layout. The trace length is $T(\sigma) = \sum_t R_t$.

Remark 5. In the SAM model, the read count is a deterministic function of the schedule and the layout. Path OSAM⁺'s own randomness (leaf positions, eviction) never changes it. With the library's deterministic layout, $T(\sigma)$ is therefore a fixed number for each σ . "Exceeding the expectation" is only meaningful once the layout is randomized, which is why the randomized layout is part of the model.

2 Structural lemmas (unconditional)

Lemma 6 (At most b items pending). *During a rebuild, $|P| \leq b$ after step 1 and after every Pack. In particular, the root never needs extra packing.*

Proof. After step 1, $|P| \leq 1 + (b - 1) = b$. If $|P| \leq b$, then $|P \circ A_j| \leq 2b - 1$. Pack then either returns at most b items unchanged, or returns $\lceil (2b - 1)/b \rceil = 2 \leq b$ chunks. $\square$

Lemma 7 (Unit depth growth). *Consider one dereference of alias x in an object. Every other alias y of that object satisfies $\text{dep}'(y) \leq \text{dep}(y) + 1$, and x itself satisfies $\text{dep}'(x) \leq \text{dep}(x)$. Aliases of other objects are unaffected. This holds for every layout, deterministic or randomized.*

Proof. Every leaf other than x lies in exactly one side subtree A_j , whose internal shape is untouched. Only its root moves. An item's new depth is 1 (child of the new root) plus the number of Packs that wrapped it under a fresh node. Each Pack wraps an item at most once.

For $j \leq D - 2$, A_j enters P at level j , so it takes part in the Packs at levels $j, j - 1, \dots, 0$. That is $j + 1$ of them, so its root's new depth is at most $j + 2$. The old depth was $j + 1$.

x' and A_{D-1} take part in all $D - 1$ Packs, so their new depth is at most D . The old depth was D . $\square$

Corollary 8 (Flat regime). *If $f \leq b$, every dereference costs exactly 2 reads, forever (exactly 1 read if $f = 1$).*

Proof. The install tree is flat, so $D = 1$. Then $P = (x', A_0)$ holds all $f \leq b$ leaves, no Pack occurs, and the new root again has all f leaves as children. Induct on the number of dereferences. $\square$

Corollary 9 (Schedule-computable range). *Fix σ and an object o with $f_o > b$. Let t be a dereference of alias a of o . Let g_t be the number of dereferences of o since a 's previous dereference t' , or since the start if there is none. Then*

$$2 \leq R_t \leq U_t(\sigma) := \begin{cases} U_{t'}(\sigma) + g_t & \text{if } t' \text{ exists,} \\ H_0(f_o) + 1 + g_t & \text{otherwise.} \end{cases}$$

Proof. Apply Lemma 7 once per intervening dereference of o , starting from the install height. The lower bound is $\text{dep} \geq 1$ for $f \geq 2$. $\square$

3 Concentration

Theorem 10 (Concentration against oblivious adversaries). *Assume a randomized layout, and fix any schedule σ . Let $O_{>}$ be the set of objects with $f_o > b$. Let n_o be the number of dereferences of o in σ , and $r_o = \sum_{t: o_t=o} (U_t(\sigma) - 2)$. Then for every $\lambda > 0$,*

$$\Pr[T(\sigma) \geq \mathbb{E}[T(\sigma)] + \lambda] \leq \exp\left(-\frac{2\lambda^2}{\sum_{o \in O_{>}} r_o^2}\right).$$

Equivalently, with probability at least $1 - e^{-s}$, $T(\sigma) \leq \mathbb{E}[T(\sigma)] + \sqrt{\frac{s}{2} \sum_{o \in O_{>}} r_o^2}$.

Proof. Write $T(\sigma) = C(\sigma) + \sum_{o \in O_{>}} T_o$. The term $C(\sigma)$ collects the objects with $f_o \leq b$, and is deterministic by Corollary 8. T_o depends only on $\sigma|_o$, on the initial bijection of o , and on the permutations drawn during o 's own rebuilds. The schedule is fixed in advance, and fresh randomness is drawn per rebuild, so these random inputs are disjoint across objects. The T_o are therefore mutually independent. (Drawing everything from one shared stream gives the same joint law, because each draw uses fresh bits.) By Corollary 9, $T_o \in [2n_o, 2n_o + r_o]$. Hoeffding's inequality for independent bounded summands finishes the proof. $\square$

Reading the bound. Suppose each object with $f > b$ is dereferenced at most once in the run ($r_o = H_0 - 1$) over K such dereferences. The excess is then at most $(H_0 - 1)\sqrt{sK/2}$, i.e. it grows like $\sqrt{K}$. With $b = 30$, $f = 952$ ($H_0 = 3$), $K = 10^4$ and $s = 40$ (failure probability 2^{-57}), the excess is at most 894 reads, on an expected total of about $4.5 \cdot 10^4$ (2%). The bound degrades for *hot* objects: an object dereferenced n times with long gaps has $r_o = \Theta(n^2)$ in the worst case. That is where Theorem 12 is needed.

Definition 11 (Influence). Let ρ_0 be the initial layout and ρ_t the randomness of the t -th dereference. The influence of step t is

$$c_t = \sup_{\rho_{<t}} \sup_{\rho_t, \rho'_t} \left| \mathbb{E}[T \mid \rho_{<t}, \rho_t] - \mathbb{E}[T \mid \rho_{<t}, \rho'_t] \right|.$$

Theorem 12 (Conditional $\sqrt{K}$ bound). *If $c_t \leq c$ for every t , then for every schedule σ with K dereferences of objects in $O_{>}$,*

$$\Pr[T(\sigma) \geq \mathbb{E}[T(\sigma)] + \lambda] \leq \exp\left(-\frac{\lambda^2}{2Kc^2}\right).$$

Proof. $M_t = \mathbb{E}[T \mid \rho_0, \dots, \rho_t]$ is a Doob martingale in which only the K steps that touch $O_{>}$ carry randomness. Its increments are bounded by $c_t \leq c$. Apply Azuma-Hoeffding. $\square$

Conjecture 13. *For the randomized layout, $c = O(b H_0(f))$, independent of the schedule and of K . Also, for every oblivious schedule, $\mathbb{E}[R_t] \leq H_0(f) + 1 + \gamma_b$ for a constant γ_b .*

The second half of the conjecture is what padding needs. The public budget has to cover $\max_{\sigma} \mathbb{E}[T(\sigma)]$ plus the slack.

4 Evidence

Concentration (`src/bin/concentration.rs`). This uses 100 independent layouts per fixed schedule and a window of k dereferences of one object after a warm-up of $2f$ dereferences. Across all $(b, f) \in \{(4, 300), (16, 952), (30, 952), (30, 3000)\}$ and all six schedules (cyclic, stride, uniform, Zipf, bursts, hot-set-then-scan), we observe:

- $\text{sd}(T_k)/\sqrt{k} \in [0.59, 1.32]$ for every schedule except the phased hot-set-then-scan one (0.81–4.13).
- The maximum over 100 layouts is at most $\mathbb{E} + 3.1 \text{ sd}$.

This is the $\sqrt{K}$ scaling of Theorem 12 with $c \approx 1$, even though a single object is dereferenced 10^4 times, far outside the regime where Theorem 10 is sharp.

Influence (`src/bin/influence2.rs`). We branch at a random step, then use common random numbers for 400 futures. The signed influence estimates for $b = 30$, $f = 952$ at look-ahead $L = 10, 100, 1000, 4000$ are below one read in absolute value at $L \leq 100$. At $L = 1000$ and 4000 they are at most 9 reads for the uniform and cyclic schedules, all within 2.3 standard errors of 0. They are larger but still noisy (≤ 22 reads) for hot-set-then-scan. Under this coupling the two copies do not re-synchronize, so the per-path differences grow like $\sqrt{L}$ while their means stay small. A proof of Conjecture 13 needs a coupling that merges two layouts once they have pushed the same set of items through a Pack.

Expectation Mean reads per dereference with the randomized layout (deterministic library in parentheses):

(b, f)	$H_0 + 1$	uniform	cyclic	Zipf	bursts
(4, 300)	6	8.11 (7.70)	8.33 (6.43)	7.04 (6.80)	2.87 (2.20)
(16, 952)	4	5.51 (5.07)	5.56 (4.79)	4.92 (4.66)	2.44 (2.04)
(30, 952)	4	4.79 (4.48)	4.82 (4.05)	4.40 (4.18)	2.34 (2.02)
(30, 3000)	4	5.55 (5.07)	5.58 (3.59)	4.95 (4.60)	2.45 (2.03)

Randomizing the layout costs 5–55% in expectation, but it turns worst-case-over-schedules padding into expectation-plus- $O(\sqrt{K})$ padding.

The deterministic potential fails (`src/bin/potential.rs`). With the library’s deterministic layout, the rank potential $\Phi = c \sum_v \log_2 s(v)$ does not certify an amortized bound of the form $R_i + \Phi_i - \Phi_{i-1} \leq O(\log f)$ for any tested scale c . Single rebuilds can raise Φ by 12–25 bits, and an adversary that always picks the deepest alias is not the worst case; cyclic scans are. This is one more reason to use the randomized layout together with an oblivious adversary.

5 The binary pointer: a deterministic bound

The binary multi-write pointer (`MultiWritePointer`) admits a much simpler and stronger statement. Its bound is deterministic, so it needs no randomized layout and holds even against *adaptive* adversaries.

Definition 14 (Binary alias tree and rebuild). The tree is full binary: every cell stores its parent and its sibling, and leaves are aliases. The install layout splits f aliases into $\lfloor f/2 \rfloor$ and $\lceil f/2 \rceil$ recursively. Dereferencing alias x maintains a state (X, Y, P) of two new subtrees and one old node with $\text{leaves}(X) \cup \text{leaves}(Y) = \text{leaves}(P)$. It starts from $(\{x'\}, \text{sib}(x), \text{par}(x))$ after reading x . Each iteration reads P :

- If P is the root, a new root receives children X and Y . Stop.
- Otherwise read $G = \text{par}(P)$, and let $S_P = \text{sib}(P)$.
 - If G is the root, form $N = (Y, S_P)$; a new root receives X and N . Stop.
 - Otherwise let $S_G = \text{sib}(G)$, form $L = (X, Y)$ and $R = (S_P, S_G)$, and continue with $(L, R, \text{par}(G))$.

Every path cell is read once, so the cost is again $\text{dep}(x) + 1$. This is path halving: two levels are consumed per iteration and one is created.

Lemma 15 (Access lemma). *Let $\Phi = \sum_v \text{internal} \log_2 s(v)$, where $s(v)$ is the number of aliases below v . One dereference in an object with f aliases satisfies*

$$R \leq 2 \log_2 f + 1 + \Phi - \Phi'.$$

Proof. Let $\mu = \log_2 s(P)$ for the current state, so $\mu \geq 1$ after the first read.

In a middle iteration, write $a = s(P)$, $b = |S_P|$ and $c = |S_G|$. The node L has exactly P 's leaves, and R replaces G , so $\Delta\Phi = \log_2(b + c) - \log_2(a + b)$. By AM–GM, $a(b + c) \leq ((a + b + c)/2)^2$, so

$$2 + \Delta\Phi \leq 2 \log_2(a + b + c) - \log_2 a - \log_2(a + b) \leq 2(\mu' - \mu),$$

where $\mu' = \log_2(a + b + c) = \log_2 s(\text{par}(G))$.

The root case costs one read, with $\Delta\Phi = 0$ and $\mu = \log_2 f$. The G -root case costs two reads, with $\Delta\Phi = \log_2(|Y| + |S_P|) - \mu \leq \log_2 f - \mu$.

Telescoping from $\mu_0 \geq 1$, the total is at most $2 \log_2 f$ in the first case and $1 + \mu + \log_2 f \leq 2 \log_2 f + 1$ in the second. $\square$

Theorem 16 (Deterministic trace bound, binary pointers). *For every sequence of k dereferences of one object, chosen adaptively or not, starting from its install layout,*

$$T \leq k(2 \log_2 f + 1) + \Delta(f), \quad \Delta(f) = B(f) - (f - 1) \leq 2f,$$

where $B(1) = 0$ and $B(f) = \log_2 f + B(\lfloor f/2 \rfloor) + B(\lceil f/2 \rceil)$ is the install potential. Over many objects, $T(\sigma) \leq \sum_t (2 \log_2 f_{o_t} + 1) + \sum_{o \text{ touched}} \Delta(f_o)$. A window that starts in an arbitrary state instead of the install layout pays at most $(f - 1)(\log_2 f - 1)$ instead of $\Delta(f)$.

Proof. Sum Lemma 15; the potentials telescope. Every internal node has $s \geq 2$ and there are $f - 1$ of them, so $\Phi \geq f - 1$. Also $\Phi \leq (f - 1) \log_2 f$. Induction gives $B(f) \leq 3f - \log_2 f - 3$, since $2 \log_2 f - \log_2(\lfloor f/2 \rfloor \lceil f/2 \rceil) \leq 2 + \log_2 \frac{4}{3} < 3$. Hence $\Delta(f) \leq 2f$. Numerically, $\Delta(f) \leq 1.06f$ for all $f \leq 2 \cdot 10^5$. $\square$

Check (src/bin/potential_binary.rs). Lemma 15 held on all 126,000 dereferences of the library's `MultiWritePointer`: $f \in \{2, \dots, 3000\}$, with random, cyclic, reverse, Zipf, bursty, and always-deepest (adaptive) schedules. Every step had at least one read of slack.

Why the r-ary rule resists this argument. The proof uses two properties of path halving: the new left node inherits its old parent’s rank exactly, and only one new node absorbs the side subtrees of two levels. The r-ary rebuild carries up to b pending items per level and re-chunks them, so it does neither. We measured single rebuilds raising Φ by 12–25 bits.

Price. Binary trees cost $\approx \log_2 f$ reads per dereference, against $\approx \log_b f$. For $f = 952$ the binary pointer’s mean is 12.9 (uniform), 6.7 (cyclic) and 2.3 (bursts) reads, and Theorem 16 certifies 20.8 per dereference. The r-ary pointer ($b = 30$) averages 4.5 on the uniform schedule, and a shape-preserving r-ary rebuild costs exactly 4.

6 What the server sees

Standard OSAM⁺ (the setting of the Concrete OSAM paper) performs every write by reading a dummy path, so it leaks the number of reads *plus* writes. There the binary pointer costs exactly $C = 3R - 1$ per dereference. The r-ary backend used here (`MULTI_WRITE_RARY`) is different. Writes are local: they go to the stash and are drained by the P eviction paths that each read downloads, with P much smaller than b (we use $P = 1$). The server sees only reads, each of $1 + P$ paths, plus public flushes during bulk builds. So R is the leaked quantity, and a large b saves both round trips and bandwidth. All the rules in this note write $O(b)$ cells per read, which is the property the backend relies on to keep the stash bounded. The alias model now also counts writes. It matches the library write for write on the same 150,000 dereferences (`validate_model`).

7 An r-ary splay rule with a proven bound: path halving

The current rule dissolves the whole path and re-chunks it, and no rank potential certifies it (Section 5). We now keep the splay behavior but restructure like the binary pointer: two path nodes per step, halving the path. The difference from the binary rule is that the carried set may hold up to b items.

The rule. Let $s(v)$ be the number of aliases below v . Write $\text{sib}(v)$ for the other children of v ’s parent. A dereference of alias a reads a ’s cell and replaces a by a fresh leaf. It then keeps a *carried set* K : the new child set of the current path node p , initially the children of a ’s parent with a replaced. Repeat:

1. Read p . If p is the root, write a new root with children K . Stop.
2. Let $S_1 = \text{sib}(p)$, and read the parent p' of p . If p' is the root, write a new root with children $K \cup S_1$ if $|K| + |S_1| \leq b$, and otherwise with children $\{\ell\} \cup S_1$, where ℓ is a new node over K . Stop.
3. Let $S_2 = \text{sib}(p')$ and p'' the parent of p' . The new child set K' of p'' is:
 - (A0) $K \cup S_1 \cup S_2$, if it has at most b items;
 - (A) otherwise $\{\ell\} \cup S_1 \cup S_2$, with $\ell = \text{node}(K)$, if that has at most b items;
 - (B) otherwise $\{\ell, r\} \cup (S_2 \setminus D)$. Here D holds the $\max(0, |S_2| + 2 - b) \leq 1$ smallest members of S_2 , and $r = \text{node}(S_1 \cup D)$, or r is that single member when $|S_1 \cup D| = 1$.

Discard p and p' , set $p \leftarrow p''$ and $K \leftarrow K'$, and repeat.

Theorem 17 (*r*-ary path halving). *Let $\Phi = \sum_{v \text{ internal}} \log_2 s(v)$ and $f \geq 2$. Every dereference satisfies:*

1. (*single read, bounded groups*) *every cell on the path is read once, the carried set and every new group have at most b members, and every node keeps at least 2 children;*
2. (*unit depth growth*) *every other alias deepens by at most one level, and the accessed alias never deepens;*
3. (*flat regime*) *if $f \leq b$, it performs exactly 2 reads and the tree stays flat;*
4. (*access lemma*) *it performs $R \leq 2 \log_2 f + 1 + \Phi - \Phi'$ reads.*

Consequently, for every k and every sequence of k dereferences of an object installed as T_0 , even one chosen adaptively,

$$\Pr \left[\sum_t R_t > k(2 \log_2 f + 1) + \Phi(T_0) - \frac{f-1}{b-1} \right] = 0.$$

Proof. (1) holds by construction. $|K| \leq b$ holds initially, and (A0), (A) and (B) each produce at most b items. In (B), $|K'| = 2 + |S_2| - |D| \leq b$ and $|S_1 \cup D| \leq (b-1) + 1$. Every new node gets at least 2 children.

(2) Let d be the depth of p . A step moves K 's members from depth $d+1$ to depth d or less. It moves S_1 from depth d to d (inside r) or to $d-1$. It moves D from $d-1$ to d , and it leaves $S_2 \setminus D$ in place. Everything a step places in K' only moves up afterwards, and the final steps do not deepen anything. Every off-path subtree occurs in exactly one S_1 or S_2 , so it deepens at most once.

(3) If $f \leq b$, the root's children are all aliases, so step 1 applies at once: 2 reads, and the tree is again flat.

(4) Let $x = s(p)$ and $\mu = \log_2 x$. Initially $\mu \geq 1$. K covers the same aliases as the old p , and $\Delta\Phi$ counts the removed and the created nodes. Step 1 performs 1 read, and $\Delta\Phi = 0$. Step 2 performs 2 reads, and $\Delta\Phi \leq 0$. In step 3, let $y = s(S_1)$, let $\delta = s(D)$, and let $\mu' = \log_2 s(p'')$, where $s(p'') \geq x + y + \delta$. The step performs 2 reads.

- (A0): $\Delta\Phi = -\log_2 x - \log_2(x+y) \leq -2$, so $2 + \Delta\Phi \leq 0$.
- (A): $\Delta\Phi = -\log_2(x+y)$. Every node has at least 2 children, so $x \geq 2$, $y \geq 1$ and $s(p'') \geq 4$. Hence $2 + \Delta\Phi \leq 2(\mu' - \mu)$ (check $x+y=3$ separately).
- (B): $\Delta\Phi = \log_2(y+\delta) - \log_2(x+y)$. By AM-GM, $x(y+\delta) \leq ((x+y+\delta)/2)^2$, so

$$2 + \Delta\Phi \leq 2 \log_2(x+y+\delta) - \log_2 x - \log_2(x+y) \leq 2(\mu' - \mu).$$

If r is a single member, $\Delta\Phi = -\log_2(x+y)$, as in (A).

Summing, the steps telescope from $\mu \geq 1$ to at most $\log_2 f$. With the leaf read and the final step, $R + \Delta\Phi \leq 1 + 2(\log_2 f - 1) + 2 = 2 \log_2 f + 1$. For the corollary, sum the lemma, and use that at the end every internal node has size at least 2 and there are at least $(f-1)/(b-1)$ of them. $\square$

The bound is deterministic and needs no randomness or assumption on the adversary. It is the *r*-ary counterpart of the binary theorem (Theorem 16), and it keeps every structural property of the current rule. For the installed tree, $\Phi(T_0) = O(f \log_2 b/b)$ is small.

We checked all four claims on 180,000 dereferences (**halving**): $b \in \{4, 8, 30\}$, $f \in \{16, 100, 952, 3000\}$, and uniform, cyclic, Zipf, bursty and deepest-alias schedules. The access lemma always held, with slack at least 6 reads. The largest depth growth of another alias was 1, and $f \leq b$ always cost 2.

The proven bound is in base 2; the data say base b . The telescoping above gains only a factor 2 in size per two reads, as in the binary pointer. With the potential $2 \sum \log_b s(v)$ instead, the largest amortized cost of a single dereference, $R + 2\Delta\Phi_b$, was:

b	f	$2\log_b f$	fixed schedules (6 kinds)	greedy adaptive adversary	$2\log_2 f + 1$
4	952	9.9	8.6	8.7	20.8
4	3000	11.6	10.4	11.1	24.1
8	952	6.6	7.0	6.7	20.8
30	952	4.0	5.0	5.0	20.8
30	3000	4.7	5.0	5.0	24.1

The greedy adversary (**halving_adv**) tries the deepest alias and 40 random ones at every step, on a clone of the tree, and dereferences the one with the largest amortized cost.

Conjecture 18. For r -ary path halving, $R \leq 2\log_b f + c + 2(\Phi_b - \Phi'_b)$ for an absolute constant c (the data suggest $c \leq 1$), where $\Phi_b = \sum_v \log_b s(v)$.

Where a proof must work. In a type-(B) step, write $u = y + \delta$ for the size of r and z for the total size of S_2 . The base- b step inequality is $s(p'')(x+y)/(xu) \geq b$. It fails in a binary-like configuration: p has one sibling subtree of about its own size ($y \approx x$), while p' has many small siblings (z small, but $|S_1| + |S_2| \geq b$). Then the step doubles the size for two reads. Such steps cost $2 - 4\log_b 2$ each, beyond the base- b bound. A proof needs a potential that also charges shapes where a node has one child much larger than the rest. Neither the fixed schedules nor the greedy adversary produced a run of such steps.

Against the current rule. Reads per dereference (mean / max, 3,000 steps):

b	f	uniform		cyclic		deepest		bursty	
		halving	current	halving	current	halving	current	halving	current
4	952	6.5/9	9.0/16	4.3/13	5.4/45	7.0/8	9.8/11	3.3/11	3.1/25
8	3000	5.3/7	7.2/12	3.5/6	3.2/35	6.0/7	7.7/9	3.2/7	2.8/17
30	952	3.1/5	4.4/8	3.1/5	2.7/10	4.0/5	4.7/6	3.0/5	2.3/11
30	3000	4.0/5	5.0/9	3.3/5	2.5/7	5.0/5	5.6/7	3.1/5	2.4/10

Halving has a lower mean on uniform and deepest-alias schedules, and a much lower maximum everywhere: its worst single dereference stays near $\log_b f + 2$. It is slower on bursts, because a re-accessed alias only moves up half the path, so it costs 3 reads rather than 2. A final step that collapses into the root whenever the carried set fits would recover part of that.

8 A balanced r -ary pointer with downward pointers

For static structures, where queries only dereference, and for any setting that needs a worst-case bound, we give up splaying and keep each alias tree *complete*. The shape is a function of the alias count f alone: all leaves are at depth $H = \lceil \log_b f \rceil$ ($H = 0$ for $f = 1$, where the alias is the root), and leaf j follows the base- b digits of j (**src/bptr.rs**).

Cells. A root holds the value, f , H and the address of its child list. Every alias and every internal node has a cell holding its parent's address and the parent's whole child group, as pairs (node, down). An internal node v also has a child-list cell $\text{Kids}(v)$ that holds the same group. This is the downward pointer. Because a member's cell is a function of its parent and its group, a dereference can rewrite every off-path member without reading it. Child lists are only rewritten during dereferences, never read, so their addresses stay put until a copy or delete reads them.

Theorem 19 (Balanced pointer). *Starting from a bulk install or from New, and after any sequence of dereferences, copies and deletes, even one chosen adaptively, the alias tree is the complete tree for the current f , and:*

1. *a dereference performs exactly $H + 1$ reads and at most $(b + 1)H + 1$ writes (at most $b + 1$ per read). It performs fewer reads only when the walk meets a root that is open in the client cache;*
2. *a copy and a delete each perform exactly $2H$ reads, or 1 when $H = 0$;*
3. *every address is read at most once, and an alias’s cell is read only by its own holder.*

Hence, for every k , $\Pr [\sum_t R_t > \sum_t (\lceil \log_b f_t \rceil + 1)] = 0$ over any k dereferences.

Proof sketch. Dereference. The walk reads the alias and each ancestor once. Each read cell gets a fresh address. For each path node v , the group of v ’s children is known, because it is stored in the cell of the child we came from. So all of v ’s children are rewritten with v ’s new address, and $\text{Kids}(v)$ is rewritten in place. No node moves, so the shape is unchanged.

Copy. After the upward walk, the new alias goes to position f . The walk goes down from the root along the digits of f . It reads $\text{Kids}(u)$ for each frontier node u that is not on the alias’s path (at most $H - 1$ of them), gives that child list a fresh address, and rewrites u ’s group. Missing frontier nodes are created. If $f = b^H$, a new level is inserted below the root.

Delete. The upward walk, then the downward walk to position $f - 1$ (again at most $H - 1$ child-list reads). The last alias is moved into the hole: its cell is rewritten in place, so its holder’s address stays valid. Emptied frontier nodes are removed. If the root is left with a single internal child, the tree is contracted, which needs at most one more child-list read, and only when the frontier walk needed none. When one alias remains, its cell becomes the root.

Copies and deletes are padded with dummy reads to exactly $2H$. Every read in these walks is of a path node or a child list, never of another holder’s alias. $\square$

The implementation passes randomized tests of every claim, with exact read counts: dereferences for $b = 2, \dots, 30$ and $f \leq 952$; 3,000 random copies, deletes, puts and gets per fanout; copies that grow the tree while its root is cached; and mixed workloads through sam-model’s client cache. With $b = 30$ and 256-byte blocks (a node cell is $6 + 8b$ bytes), SPARQ-OSAM’s per-query budgets now hold with raw fan-in. On seven datasets, with 500 queries per tail, all answers were correct and no query exceeded its budget.

Compared with the splay rules of Sections 5–7, the balanced pointer gives up adaptivity and zero-read copies. In return it has exact worst-case costs and no trace variance from the pointer state.